



Redis Notes

Node.js Backend — Complete Reference

Data types, real-world use cases, Node.js code examples, and architecture patterns for using Redis effectively in a Node.js / Express backend.

1. Redis Data Types

Redis is an in-memory data store that stores data primarily in RAM. It is commonly used for caching, sessions, counters, rate limiting, queues, pub/sub, real-time features, and other high-speed operations.

1. Strings

DEFINITION

String is the simplest and most commonly used Redis data type. It stores a sequence of bytes, which can represent text, numbers, JSON strings, etc.

A Redis key can store one string value.

COMMON FUNCTIONS

Command	Purpose
SET	Store a value
GET	Get a value
SETEX	Set value with expiration
EXPIRE	Set expiration on existing key
TTL	Check remaining expiry time
DEL	Delete key
INCR	Increment number
DECR	Decrement number
INCRBY	Increment by specific amount
APPEND	Append text
MGET	Get multiple values
MSET	Set multiple values
EXISTS	Check whether key exists

SIMPLE NODE.JS CODE

```
import Redis from "ioredis";

const redis = new Redis();

await redis.set("name", "Saurabh");

const name = await redis.get("name");

console.log(name);
```

With expiration:

```
await redis.set("otp", "123456", "EX", 60);
```

The OTP will automatically expire after 60 seconds.

Counter:

```
await redis.set("views", 10);
```

```
await redis.incr("views");

console.log(await redis.get("views"));
```

REAL-LIFE EXAMPLE

OTP storage — `otp:user123 → 493821`

Store the OTP for 60 seconds. After 60 seconds Redis automatically removes it.

2. Arrays

DEFINITION

Redis does not have a native data type called Array like JavaScript.

When people talk about storing arrays in Redis, they usually mean either:

- Redis Lists
- JSON arrays using RedisJSON

For example, using RedisJSON:

```
["Node.js", "Redis", "MongoDB"]
```

SIMPLE NODE.JS CODE

Using JSON:

```
const skills = ["Node.js", "Redis", "MongoDB"];

await redis.set("skills", JSON.stringify(skills));

const data = JSON.parse(await redis.get("skills"));

console.log(data);
```

REAL-LIFE EXAMPLE

A user's recently searched keywords:

`recentSearches:user123 → ["redis", "nodejs", "mongodb"]`

3. Lists

DEFINITION

A List is an ordered collection of strings. Redis Lists are implemented as linked structures and are useful when you need insertion/removal from the beginning or end.

COMMON FUNCTIONS

Command	Purpose
LPUSH	Add to left
RPUSH	Add to right
LPOP	Remove from left
RPOP	Remove from right
LRANGE	Get elements in range
LLEN	Get list length

LREM	Remove elements
------	-----------------

SIMPLE NODE.JS CODE

```
await redis.rpush("tasks", "task1");
await redis.rpush("tasks", "task2");
await redis.rpush("tasks", "task3");

const tasks = await redis.lrange("tasks", 0, -1);

console.log(tasks);
```

Output:

```
["task1", "task2", "task3"]
```

Queue example:

```
await redis.lpush("queue", "job1");

const job = await redis.rpop("queue");

console.log(job);
```

REAL-LIFE EXAMPLE

Job Queue

```
queue
  ↓
job1
job2
job3
```

Workers can take jobs from the queue one by one.

4. Geospatial

DEFINITION

Redis Geospatial allows you to store locations using longitude and latitude and perform geographical searches.

COMMON FUNCTIONS

Command	Purpose
GEOADD	Add location
GEOPOS	Get coordinates
GEODIST	Calculate distance
GEOSEARCH	Search nearby locations

SIMPLE NODE.JS CODE

```
await redis.geoadd(
  "restaurants",
  77.5946,
  12.9716,
  "restaurant1"
);
```

```
const result = await redis.geosearch(
  "restaurants",
  "FROMLONLAT",
  77.59,
  12.97,
  "BYRADIUS",
  5,
  "km"
);

console.log(result);
```

REAL-LIFE EXAMPLE

Food delivery app — User wants restaurants within 5 km.

```
User location
  ↓
Redis GEOSEARCH
  ↓
Restaurants within 5 km
```

5. Hashes

DEFINITION

A Hash stores multiple field-value pairs inside one Redis key. It is similar to a JavaScript object.

```
user:101
  name → Saurabh
  age → 22
  city → Delhi
```

COMMON FUNCTIONS

Command	Purpose
HSET	Set field
HGET	Get field
HGETALL	Get all fields
HDEL	Delete field
HEXISTS	Check field
HINCRBY	Increment field

SIMPLE NODE.JS CODE

```
await redis.hset("user:101", {
  name: "Saurabh",
  age: 22,
  city: "Delhi"
});

const user = await redis.hgetall("user:101");

console.log(user);
```

REAL-LIFE EXAMPLE

User session

```
session:abc123

userId → 101
name   → Saurabh
role   → user
```

Instead of creating multiple keys, related information can be stored together.

6. Sets

DEFINITION

A Set is an unordered collection of unique values. Duplicates are automatically removed.

```
{Node.js, Redis, MongoDB}
```

COMMON FUNCTIONS

Command	Purpose
SADD	Add member
SMEMBERS	Get members
SREM	Remove member
SISMEMBER	Check membership
SCARD	Get number of members
SINTER	Intersection
SUNION	Union

SIMPLE NODE.JS CODE

```
await redis.sadd("skills", "Node.js");
await redis.sadd("skills", "Redis");
await redis.sadd("skills", "Node.js");

console.log(await redis.smembers("skills"));
```

Node.js will appear only once.

REAL-LIFE EXAMPLE

Unique visitors

```
visitedUsers
→ user101
→ user102
→ user103
```

If user101 visits again, the Set doesn't create a duplicate.

7. Sorted Sets

DEFINITION

A Sorted Set is a collection of unique members where every member has a score. Redis automatically sorts members according to their score.

score → member

100 → Rahul
90 → Saurabh
80 → Aman

COMMON FUNCTIONS

Command	Purpose
ZADD	Add member with score
ZRANGE	Get members
ZREVRANGE	Get in reverse order
ZSCORE	Get score
ZREM	Remove member
ZRANK	Get ranking

SIMPLE NODE.JS CODE

```
await redis.zadd("leaderboard", 100, "Rahul");
await redis.zadd("leaderboard", 90, "Saurabh");
await redis.zadd("leaderboard", 80, "Aman");

const leaderboard = await redis.zrevrange(
  "leaderboard",
  0,
  2,
  "WITHSCORES"
);

console.log(leaderboard);
```

REAL-LIFE EXAMPLE

Gaming leaderboard

1000 → Player A
950 → Player B
900 → Player C

When a player gets more points, update their score:

```
await redis.zincrby("leaderboard", 50, "Saurabh");
```

Redis automatically changes their position.

8. Streams

DEFINITION

Redis Streams are an append-only data structure used for storing and processing a sequence of events. They are useful for event-driven systems and message processing.

COMMON FUNCTIONS

Command	Purpose
XADD	Add event

XRANGE	Read events
XREAD	Read stream
XGROUP	Create consumer group
XREADGROUP	Read using consumer group

SIMPLE NODE.JS CODE

```
await redis.xadd(
  "orders",
  "*",
  "orderId",
  "101",
  "userId",
  "50"
);

const events = await redis.xrange("orders", "-", "+");

console.log(events);
```

REAL-LIFE EXAMPLE

Order processing

```
Order Created
  ↓
Redis Stream
  ↓
Payment Service
  ↓
Inventory Service
  ↓
Notification Service
```

Every event can be stored and processed by consumers.

9. JSON

DEFINITION

RedisJSON is a Redis module that allows Redis to store and manipulate JSON documents directly. Instead of converting JSON into a string manually, you can work with JSON structures.

Example:

```
{
  "name": "Saurabh",
  "age": 22,
  "skills": ["Node.js", "Redis"]
}
```

SIMPLE NODE.JS CODE

With a Redis client that supports RedisJSON:

```
await redis.json.set("user:101", "$", {
  name: "Saurabh",
  age: 22,
  skills: ["Node.js", "Redis"]
});

const user = await redis.json.get("user:101");
```



```
console.log(user);
```

REAL-LIFE EXAMPLE

Product cache

```
product:101
{
  name: "Laptop",
  price: 60000,
  category: "Electronics"
}
```

Useful when your application frequently needs complete JSON documents.

10. Vector Sets

DEFINITION

Vector Sets are designed to store vectors/embeddings and perform vector similarity search. They are useful in modern AI applications.

```
"Node.js backend"
  ↓
Embedding
  ↓
[0.21, 0.72, 0.13, ...]
```

Redis can compare vectors and find similar items.

SIMPLE EXAMPLE

Conceptually:

```
// Store an embedding/vector
await redis.vadd(
  "documents",
  [0.21, 0.72, 0.13],
  "doc:101"
);
```

Then search for similar vectors.

```
const results = await redis.vsim(
  "documents",
  [0.20, 0.70, 0.15],
  5
);

console.log(results);
```

Exact vector-set commands/API syntax can vary with the Redis version/client being used, so check the client documentation for the version you're running.

REAL-LIFE EXAMPLE

AI semantic search — User searches: "How does Node.js handle async operations?"
The application converts it into an embedding and Redis finds documents with similar vectors.

11. Time Series

DEFINITION

Redis Time Series is designed for storing time-stamped numerical data. It is useful when data changes over time.

timestamp	CPU
10:00	30
10:01	35
10:02	42

SIMPLE EXAMPLE

Conceptually:

```
await redis.ts.add(
  "server:cpu",
  "*",
  42
);
```

Then retrieve historical data:

```
const data = await redis.ts.range(
  "server:cpu",
  "-",
  "+"
);

console.log(data);
```

REAL-LIFE EXAMPLE

Server monitoring

```
Server
  ↓
CPU usage
  ↓
Redis Time Series
  ↓
Monitoring dashboard
```

You can store CPU, memory, temperature, request rate, etc.

12. Pub/Sub in Redis

DEFINITION

Redis Pub/Sub allows one application to publish messages to a channel while other applications subscribe to that channel. It follows:

```
Publisher
  ↓
Redis Channel
  ↓
Subscriber
```

The publisher and subscriber don't directly communicate with each other.

PUBLISHER

```
import Redis from "ioredis";

const redis = new Redis();

await redis.publish(
  "notifications",
  "New order received"
);
```

SUBSCRIBER

Subscriber needs a separate Redis connection.

```
import Redis from "ioredis";

const subscriber = new Redis();

await subscriber.subscribe("notifications");

subscriber.on("message", (channel, message) => {
  console.log(channel);
  console.log(message);
});
```

REAL-LIFE EXAMPLE

Chat application

```

User A
  ↓
Publisher
  ↓
Redis channel: chat
  ↓
Subscribers
  ↓
User B
User C
User D
```

When User A sends a message, all interested subscribers receive it.

Important Point: Redis Pub/Sub is generally not a durable message queue. If a subscriber is disconnected when a message is published, it won't receive that old message. For durable event processing, Redis Streams are more appropriate.

13. How to Actually Implement Redis in Node.js

DEFINITION

In a Node.js backend, Redis is usually connected as a separate data store. Typical architecture:

```
Client
  ↓
Node.js / Express
  ↓
Redis
  ↓
MongoDB / PostgreSQL
```

STEP 1 — INSTALL REDIS CLIENT

Using ioredis:

```
npm install ioredis
```

STEP 2 — CREATE REDIS CONNECTION

```
import Redis from "ioredis";

const redis = new Redis(
  process.env.REDIS_URL || "redis://localhost:6379"
);

redis.on("connect", () => {
  console.log("Redis connected");
});
```

STEP 3 — USE REDIS

Example: caching users.

```
app.get("/users/:id", async (req, res) => {
  const id = req.params.id;

  // Check Redis first
  const cachedUser = await redis.get(`user:${id}`);

  if (cachedUser) {
    return res.json(JSON.parse(cachedUser));
  }

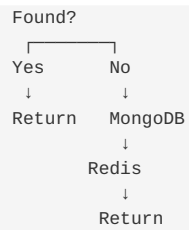
  // Get from database
  const user = await User.findById(id);

  // Store in Redis
  await redis.set(
    `user:${id}`,
    JSON.stringify(user),
    "EX",
    300
  );

  res.json(user);
});
```

FLOW

```
Request
  ↓
Check Redis
  ↓
```



REAL-LIFE EXAMPLE

Suppose your /products API gets 10,000 requests/minute. Instead of querying MongoDB every time:

```
10,000 requests
  ↓
Redis Cache
  ↓
Most requests served here
  ↓
MongoDB receives fewer queries
```

This can reduce database load and improve response latency.

14. Where Should You Use Redis?

The most important skill isn't memorizing Redis commands. It's knowing: *"When should I use Redis?"* Here are the major use cases.

Use Case	Redis Feature
API caching	Strings
Session storage	Hashes / Strings
OTP	Strings + TTL
Rate limiting	Strings
Counters	INCR
Leaderboards	Sorted Sets
Unique users	Sets
Job queues	Lists / Streams
Real-time messaging	Pub/Sub
Event processing	Streams
Nearby locations	Geospatial
AI semantic search	Vector Sets
Metrics	Time Series
Temporary data	Strings + TTL

15. How to Identify a Redis Use Case?

Ask these questions when designing a backend.

QUESTION 1: DOES THE DATA NEED TO BE EXTREMELY FAST?

If yes → Redis may be useful.

QUESTION 2: IS THE DATA TEMPORARY?

Examples: OTP, Session, Cache, Verification token. Redis is a strong candidate.

QUESTION 3: AM I REPEATEDLY QUERYING THE DATABASE FOR THE SAME DATA?

Example: `GET /products` — if thousands of users request the same data:

```
MongoDB
↑
|
Repeated queries
```

Use:

```
Node.js
↓
Redis Cache
↓
MongoDB
```

QUESTION 4: DO I NEED A COUNTER?

Examples: views, likes, downloads, API requests.

```
await redis.incr("video:101:views");
```

QUESTION 5: DO I NEED RANKING?

Use Sorted Sets. Examples: gaming leaderboard, top creators, top products, trending posts.

QUESTION 6: DO I NEED REAL-TIME COMMUNICATION?

Consider: Pub/Sub

QUESTION 7: DO I NEED RELIABLE EVENT PROCESSING?

Consider: Redis Streams

Important Rule: Don't use Redis simply because "Redis is faster." First ask: what problem am I solving? Redis should have a clear role in your architecture.

16. How to Run Redis?

There are several ways.

METHOD 1 — DOCKER

This is one of the easiest approaches for development.

```
docker run -d \
  --name redis \
  -p 6379:6379 \
  redis:latest
```

Check:

```
docker ps
```

Connect from Node.js:

```
const redis = new Redis("redis://localhost:6379");
```

Architecture:

```
Node.js
  ↓
localhost:6379
  ↓
Docker Redis
```


17. Redis with Docker Compose

For a real backend project, Compose is convenient.

```
services:
  redis:
    image: redis:latest
    container_name: redis
    ports:
      - "6379:6379"
```

Run:

```
docker compose up -d
```

Stop:

```
docker compose down
```

IF NODE.JS IS ALSO INSIDE DOCKER

Don't use:

```
redis://localhost:6379
```

Instead use the service name:

```
redis://redis:6379
```

Because inside Docker:

```
Node container
  ↓
redis:6379
  ↓
Redis container
```

localhost would refer to the Node.js container itself.

18. Other Ways to Run Redis

LOCAL INSTALLATION

You can install Redis directly on your machine.

```
Node.js
↓
localhost:6379
↓
Redis
```

CLOUD REDIS

For production, you can use a managed Redis service.

```
Node.js Server
↓
Internet
↓
Managed Redis
```

The provider handles things such as infrastructure, availability, backups, scaling, etc., depending on the service.

19. Most Important Redis Concepts for a Node.js Backend Developer

If you're learning Redis specifically for backend development, focus on these in this order:

1. Redis fundamentals
↓
2. Strings
↓
3. TTL / EXPIRE
↓
4. Hashes
↓
5. Lists
↓
6. Sets
↓
7. Sorted Sets
↓
8. Pub/Sub
↓
9. Caching
↓
10. Rate Limiting
↓
11. Sessions
↓
12. Redis Streams
↓
13. Redis + Docker
↓
14. Redis + Node.js
↓
15. Redis in System Design